ICS 46 · Data Structure Implementation and Analysis · Spring 2026

PA 0 - C++ Review - Analyzing Menu Files

A C++ warm-up. These files come from AI/statistics problems, but you need none of that background — everything you need is right here.

You're writing software for a **Menu Planner**: a tool a restaurant uses to design great multi-course menus. In this first assignment you'll write a class that **reads a menu file and reports a few facts about it**. It's a review of the C++ you'll lean on all quarter: classes, pointers and dynamic memory, references and `const`, and a simple iterator.

At a Glance

- **Submit on Gradescope:** a single header-only file, `MenuModel.hpp`.
- **Build:** four small types — two `structs` and two classes — using raw dynamic arrays and the **rule of five**.
- **Graded on:** memory safety first (a leak scores 0), then your methods' *return values* checked against the reference.

WHAT A MENU FILE DESCRIBES

A `.menu` file describes one menu-design problem:

- **courses** — the parts of the meal to decide (Main, Drink, Dessert, ...). Courses are numbered `0, 1, 2, ...`.
- **dishes** — for each course, how many options it has (its dishes are numbered `0, 1, ...`).
- **pairing tables** — small score tables over one or a few courses. Each table lists **which courses it covers** (its *scope*) and a **score** for every combination of those courses' dishes.

A companion `.chosen` file (optional) lists courses that have already been **chosen** in advance.

THE .MENU FORMAT

All tokens are whitespace-separated; the blank lines are just for readability. For the tiny example `menu3`:

```
MENU              <- a format marker; read it and ignore it

3                 <- number of courses
3 2 2             <- dishes per course (course 0 has 3, courses 1 and 2 have 2)

3                 <- number of pairing tables
1 0               <- table 0 covers 1 course:  course 0
2 0 1             <- table 1 covers 2 courses: courses 0 and 1
2 1 2             <- table 2 covers 2 courses: courses 1 and 2

3                 <- table 0 has 3 score entries...
5 3 2             <- ...the scores

6                 <- table 1 has 6 entries...
4 1 1 4 3 3
```

```
4               <- table 2 has 4 entries...
3 1 1 3
```

So a `.menu` file has **four** blank-line-separated sections: the `MENU` **tag**; the **courses** (how many, then dishes per course); the **pairing scopes** (how many tables, then which courses each covers); and the **pairing scores** (each table's entry count and scores).

A table's scores are just a flat list, **one per dish-combination**, in a fixed order: the **first** course in the scope changes slowest. Writing table `1` (over course `0` Main and course `1` Drink) out as a table makes that order plain — its **score** column read top-to-bottom is exactly the list `4 1 1 4 3 3`:

Main (course 0)	Drink (course 1)	score
0	0	4
0	1	1
1	0	1
1	1	4
2	0	3
2	1	3

THE .CHOSEN FORMAT

One line: a count, then that many `course dish` pairs. `1 1 1` means "1 course chosen: course 1 is dish 1." (`menu3.chosen` is exactly that.)

WHAT YOU BUILD

Put everything in `MenuModel.hpp` (header-only — that is the only file you submit). The provided `driver.cpp` (do not modify) builds a `MenuModel` from a `.menu` file, calls its methods, and prints the report below. Keep the method names/signatures in the starter exactly.

The starter already declares all four types below: the two **structs** are complete, and the two **classes** are scaffolded for you (members, accessors, the iterator, and the `swap`/move/assignment are provided). You fill in only the parts marked `// TODO`. The four types are:

1. `struct Course { int courseIdx; int numDishes; };` — one course: its index (its position `0, 1, 2, ...`) and how many dishes it has.
2. `struct ChosenCourse { int courseIdx; int dish; };` — one chosen course.
3. `class PairingFactor` — one pairing table. It **owns** two dynamically-allocated arrays: the **scope** (a `Course const*` array — non-owning pointers to the courses it covers) and the **score table** (`double*`).
4. `class MenuModel` — the whole model. It constructs and **owns** the `Courses` (a `Course*` array), the pairing tables, and the chosen courses; each `PairingFactor`'s scope just **points into** this `Course` array. It parses the files and can be iterated over its tables.

THE REPORT

The provided `driver.cpp` builds a `MenuModel` from a `.menu` file, calls your methods, and prints a report. The **format** (these labels, in this order) is fixed; the **numbers** depend on the input file. The example below is what the driver prints for `menu3.menu` (a sample in `starter/samples/`) — a different `.menu` gives different numbers in the same format:

```
courses: 3
dishes_per_course: 3 2 2
tables: 3
table_scopes: (0) (0 1) (1 2)
table_sizes: 3 6 4
largest_table: 6
complete_meals: 12
```

If you also pass a `.chosen` file — here `menu3.chosen` — the driver appends three more lines (again, the values are for this particular file):

```
chosen_courses: 1
chosen: (1 1)
free_courses: 2
```

- `table_sizes` is each table's number of score entries; `largest_table` is the biggest.
- `complete_meals` = the product of all the dish counts (how many complete meals exist).
- `chosen` lists each chosen `(course dish)`; `free_courses` = courses − chosen.

RULES

- **Header-only.** Everything in `MenuModel.hpp`. Do not modify `driver.cpp`.
- **Standard library for input only** (`<fstream>`, `<string>`, `<stdexcept>`, `<utility>`).
- **Use raw dynamic arrays** (`new[]` / `delete[]`) for the model's data. **Do not use** `std::vector`, `std::map`, or other containers/smart pointers — this assignment is about managing memory yourself. (The autograder enforces this: it allows only the four headers above, and using an STL container scores 0. Your own helper `struct`s/classes are fine.)
- Because your classes own raw memory, give each a correct **rule of five** (destructor, copy constructor, move constructor, copy/move assignment). The starter provides the `swap` + move + assignment via the **copy-and-swap** idiom; you write the destructor and the deep-copy constructor. **No memory leaks and no double-frees.**
- **Who owns the `Course`s:** the `MenuModel` owns them; a `PairingFactor`'s scope only holds `Course const*` pointers into that array. So a factor's destructor frees its pointer array but **never** the `Course`s, and when you copy a `MenuModel` you must **re-point** each copied factor's scope into the new model's `Course`s (the starter explains where).
- **Private data members start with a leading underscore** (e.g. `_scope`), as in the starter.

BUILD & TEST LOCALLY

```
cd starter
g++ -std=c++20 -O2 -Wall -Wextra driver.cpp -o analyze
./analyze samples/menu3.menu samples/menu3.chosen
diff <(./analyze samples/menu3.menu samples/menu3.chosen) samples/expected_menu3.txt
```

`samples/` has a few `.menu`/`.chosen` files with their `expected_*.txt` outputs. Check for leaks with `-fsanitize=address,undefined`.

> **Grading environment.** Your code is compiled and run on Gradescope in **Ubuntu 22.04** with `g++` **11** at `-std=c++20` (the same standard as the command above). Make sure `MenuModel.hpp` **compiles cleanly and runs in that environment** — code that builds only on a different OS, compiler, or C++ standard (for example macOS/Apple Clang, MSVC, or an older standard) may fail to grade even if it works on your machine. The commands above mirror the autograder, so if it builds and runs there, it will build for grading.

HOW IT'S GRADED

The autograder generates many menu files of different shapes (chains, fully-connected, with chosen courses, varying sizes, . . .) and grades in two stages:

1. **Memory safety first.** Your class is built and exercised through the full rule of five (copy, move, assignment, destruction) under a memory sanitizer. **Any leak, double-free, or invalid access scores 0 for the whole assignment** — the functional tests are skipped until it's clean. (This runs in a separate build so it can't affect anything else.)
2. **Then your methods' return values.** The grader builds a `MenuModel` from each generated file and **calls your methods directly**, comparing each return value to the reference solution's — `numCourses()`, `dishes(i)`, every table's `scopeSize()`/`courseAt(j)`/`numEntries()`/`scoreAt(k)`, `completeMeals()`, and the chosen-course accessors. It is **your return values that are checked, not your printed output**, so `driver.cpp` is only for your own local testing.

> **Every test is fully shown to you** — the exact menu file and a per-method *expected vs. your return* comparison — so you can always see what happened. There are no hidden tests; only the generator that makes the files is kept on the autograder. Some tests use **freshly randomized** menu files, so re-running the autograder may grade you on different inputs — a correct solution passes them all regardless.

> **Important:** Know your own code. You may be **asked about your implementation** in lecture — to explain some aspect of how it works, or an analysis of it — so make sure you understand what you wrote, what it does, and why.

SUBMIT

What to Turn In

Submit on **Gradescope** (the **PA 0** assignment for this course). Upload **only** the file `MenuModel.hpp` — the filename must match **exactly** (the autograder opens `MenuModel.hpp`), and upload the `.hpp` file itself, not a folder or a `.zip`. You may resubmit as often as you like before the deadline; the autograder runs on each submission and shows you the full per-test results.